

The `toolsSettings` parameter provides fine-grained configuration and security controls for specific tools, allowing you to customize tool behavior and enforce restrictions at the individual tool level. It accepts a JSON object where keys are tool names (built-in tools like `"write"` or MCP tools like `"@server_name/tool_name"`) and values are tool-specific configuration objects that can include path restrictions (`"allowedPaths"`, `"deniedPaths"`), command filtering (`"allowedCommands"`, `"deniedCommands"`), service limitations (`"allowedServices"`, `"deniedServices"`), and other tool-specific settings.

This parameter acts as the final security layer, enforcing deny-before-allow rules and enabling precise control over what files can be accessed, which commands can be executed, or what AWS services can be used, even for tools that are listed in both `tools` and `allowedTools`.

Pattern Type of each tools:

| Tool              | Pattern Type       | Engine           | Features                          |
|-------------------|--------------------|------------------|-----------------------------------|
| shell/execute_cmd | Full Regex         | Rust regex crate | All regex features, auto-anchored |
| read              | Glob Patterns      | globset crate    | Shell wildcards, ** recursion     |
| write             | Glob Patterns      | globset crate    | Shell wildcards, ** recursion     |
| aws               | Exact String Match | Vec::contains()  | No patterns, exact strings only   |

Objective

Learn how to use `toolsSettings` to implement fine-grained security controls with allowed and denied patterns.

Instructions

Step 1: Create Agent with File System Restrictions

```
1 cat > .kiro/agents/restricted-dev.json << 'EOF'
2 {
3   "name": "restricted-dev",
4   "description": "Development agent with file system restrictions",
5   "prompt": "You are a development assistant with restricted file access for security.",
6   "tools": ["read", "write"],
7   "allowedTools": ["read", "write"],
8   "toolsSettings": {
9     "read": {
10      "allowedPaths": ["**/src/**", "**/docs/**", "*.md"],
11      "deniedPaths": ["**/.env*", "**/secrets/**", "*.key"]
12    },
13    "write": {
14      "allowedPaths": ["**/src/**", "**/docs/**"],
15      "deniedPaths": ["**/.env*", "**/secrets/**", ".git/**"]
16    }
17  }
18 }
19 EOF
```

Step 2: Create Agent with Command Restrictions

```
1 cat > .kiro/agents/safe-executor.json << 'EOF'
2 {
3   "name": "safe-executor",
4   "description": "Agent with safe command execution restrictions",
5   "prompt": "You are a development assistant that can run safe commands only.",
6   "tools": ["shell"],
7   "allowedTools": ["shell"],
8   "toolsSettings": {
9     "shell": {
10      "allowedCommands": ["ls .*", "cat .*", "echo .*", "npm test.*"],
11      "deniedCommands": ["df.*", "ls -lrt", "chmod 444 .*", "git push.*"]
12    }
13  }
14 }
15 EOF
```

Step 3: Create Agent with AWS Service Restrictions

```
1 cat > .kiro/agents/aws-restricted.json << 'EOF'
2 {
3   "name": "aws-restricted",
4   "description": "Agent with AWS service restrictions",
5   "prompt": "You are an AWS assistant with limited service access for safety.",
6   "tools": ["aws"],
7   "allowedTools": ["aws"],
8   "toolsSettings": {
9     "aws": {
10      "allowedServices": ["s3", "s3api", "lambda", "dynamodb"],
11      "deniedServices": ["iam", "organizations", "ce"]
12    }
13  }
14 }
15 EOF
```

Step 4: Test File System Restrictions

1. Start custom agent

```
1 kiro-cli chat --agent restricted-dev
```

2. Test Allowed Paths:

|                                                |                                     |
|------------------------------------------------|-------------------------------------|
| "Read the README.md file"                      | # Should work (*.md` allowed)       |
| "Read a file in the src directory"             | # Should work (**/src/**` allowed)  |
| "Read a file in the docs directory"            | # Should work (**/docs/**` allowed) |
| "Create a test python file in src/components/" | # Should work (**/src/**` allowed)  |

3. Test Denied Paths:

- Create test files first

```
1 touch .env.local
2 mkdir -p secrets
3 touch secrets/api-key.txt
4 touch private.key
```

- Try these prompts with the same agent:

|                                          |                                             |
|------------------------------------------|---------------------------------------------|
| "Read the .env.local file"               | # Should be blocked (**/.env*` denied)      |
| "Read secrets/api-key.txt"               | # Should be blocked (**/secrets/**` denied) |
| "Read the private.key file"              | # Should be blocked (*.key` denied)         |
| "Create a file in the secrets directory" | # Should be blocked (**/secrets/**` denied) |

Step 5: Test Command Restrictions

1. Start custom agent

```
1 kiro-cli chat --agent safe-executor
```

2. Test Safe Commands:

|                              |                                      |
|------------------------------|--------------------------------------|
| "Run 'ls -la' to list files" | # Should work (ls .*` allowed)       |
| "Run 'echo hello world'"     | # Should work (echo .*` allowed)     |
| "Run 'cat README.md'"        | # Should work (cat .*` allowed)      |
| "Run 'npm test' command"     | # Should work (npm test .*` allowed) |

3. Test Dangerous Commands:

🕒 To avoid conflicts with Kiro's default guardrails, general commands will be used instead of potentially harmful commands.

|                                   |                                                                |
|-----------------------------------|----------------------------------------------------------------|
| "Run 'df -h' to check disk usage" | # Should be blocked (df.*` denied)                             |
| "Run 'ls -lrt'"                   | # Should be blocked (denied have higher priority than allowed) |
| "Run 'chmod 444 README.md'"       | # Should be blocked (chmod 444 .*` denied)                     |
| "Run 'git push origin main'"      | # Should be blocked (git push .*` denied)                      |

Step 6: Test AWS Service Restrictions

1. Start custom agent

```
1 kiro-cli chat --agent aws-restricted
```

2. Test Allowed Services:

|                         |                                  |
|-------------------------|----------------------------------|
| "List S3 buckets"       | # Should work (s3 allowed)       |
| "List Lambda functions" | # Should work (lambda allowed)   |
| "List DynamoDB tables"  | # Should work (dynamodb allowed) |

3. Test Denied Services:

|                              |                                            |
|------------------------------|--------------------------------------------|
| "List IAM users"             | # Should be blocked (iam denied)           |
| "Check billing information"  | # Should be blocked (billing denied)       |
| "List organization accounts" | # Should be blocked (organizations denied) |

Verification

- ✔ Allowed paths and commands work without issues
- ✔ Denied paths and commands are properly blocked
- ✔ Error messages clearly indicate permission denials
- ✔ AWS service restrictions work correctly
- ✔ Security boundaries are enforced consistently

Key Learnings

- `toolsSettings` provides fine-grained control over tool behavior
- `allowedPaths/allowedCommands/allowedServices` define what's permitted
- `deniedPaths/deniedCommands/deniedServices` define what's blocked
- Deny rules are evaluated before allow rules
- Wildcard patterns (\*, \*\*) work in path and command restrictions
- Security controls help prevent accidental or malicious actions